
Feedback or Annealing?

A Controlled Study of Adaptive State Aggregation

Sunny Zhang
University of Toronto
ssunny.zhang@mail.utoronto.ca

Abstract

Adaptive state aggregation replaces full-state Bellman updates with updates on groups of states with similar estimated values. A natural rule makes these groups narrower as the Bellman residual shrinks. Any improvement, however, may come simply from narrowing the groups over time rather than from using the residual as feedback. I separate these effects on a 9,261-state inventory control problem. After each full-state sweep, the Bellman operator reduces the residual range to at most γ times its previous value. The feedback rule therefore narrows the groups at a roughly geometric rate. This observation suggests a preplanned baseline whose rate is set by γ and the solver’s update schedule, without tuning it to the observed results. I compare the methods under three mixtures of full-state and grouped updates, 20 paired random seeds, and five compute budgets. Whenever a 95% bootstrap interval excludes zero, feedback’s largest advantage over the preplanned schedule is 0.2% of the baseline error, while its largest disadvantage is 10.8%. After 10,000 iterations, every interval both contains zero and lies within the preregistered ± 0.02 range of negligible differences. The narrowing schedule matters much more than feedback itself: relative to a fixed $\varepsilon = 0.05$, it changes error by about 23% at one matched budget, helping early but hurting later. A comparison only at the end would miss both effects. Full-state value iteration also reaches the reference tolerance with fewer counted Bellman backups than any aggregation method. This case study shows that evaluations of adaptive dynamic programming should include fixed-width and matched preplanned baselines, measure policy quality, and compare more than one compute budget.

1 Introduction

State aggregation reduces the cost of dynamic programming by solving a smaller approximation to a Markov decision process (MDP) [1–3]. In the adaptive method of Chen et al. [4], states with similar current values share a group. Updates of every state alternate with cheaper updates that sample one state from each group. A fixed group width ε creates a familiar cost–accuracy tradeoff: wide groups are cheap but introduce approximation error, while narrow groups preserve accuracy but require more updates.

This suggests an adaptive rule. The Bellman residual measures how much the current value estimates would change after another full update. When it is large, wide groups may be sufficient. As it shrinks, the algorithm can use narrower groups. The central question is whether the residual supplies useful information. Because the rule also narrows the groups over time, comparing it only with a fixed width cannot show whether feedback helps or whether a preplanned schedule with the same rate would work just as well.

I test the value of the residual signal by comparing it with a geometric schedule set in advance. For each update mix, the comparison matches the starting width, minimum width, and number of

cycles needed to reach that minimum. The two methods differ only in whether the residual sets the width. The original experiment was preregistered for one update mix and a measurement at 20 ms. I retain that inconclusive result and add comparisons at fixed amounts of computation for three update mixes. The results have three main lessons. First, the standard contraction property of the Bellman operator places a geometric upper bound on the feedback schedule (Proposition 1). Thus, a geometric comparison follows from the feedback rule rather than being an arbitrary choice. Second, feedback provides no improvement at any of five compute budgets. Third, narrowing the groups over time does matter: at one matched budget it changes error by about 23% relative to a fixed width, but the direction changes as the budget grows. A comparison only at the end would reveal neither effect.

2 Related work

Classical aggregation assigns one value to every state in a group. Its value and policy errors depend on how much the true values vary within each group [1, 3, 6]. Several methods change the groups during computation. Bertsekas and Castanon [2] group states by residual size, Singh et al. [7] adjust partial group membership, variable-resolution control divides regions using value, policy, influence, or variance [8], and progressive disaggregation divides regions as the value function changes [9]. I use the simpler method of Chen et al. [4], which repeatedly groups states by current value and has an asymptotic error bound of $2\varepsilon/(1 - \gamma)$.

Bellman residuals give computable error bounds [10]. Prior work has directly minimized the range of these residuals in approximate value iteration [11], while prioritized sweeping uses local errors to choose which states to update [12, 13]. A small residual, however, does not necessarily imply a good policy [14], so I also measure policy loss. Here the range of the residual changes only one quantity, the group width. It does not change group membership rules, state priorities, or the fitting objective. I do not propose a new state representation or a new convergence theorem.

3 Method

For a finite discounted cost MDP, the Bellman update T maps the current value function V to

$$(TV)(s) = \min_a \left[c(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]. \quad (1)$$

The solver groups values $V(s)$ into nonempty intervals of width ε . A full-state iteration evaluates (1) for every state. A grouped iteration samples one state from each group, evaluates one Bellman update for that state, and uses the result to update the shared group value with step size $\alpha_t = 1/\sqrt{t}$. A schedule (ℓ_g, ℓ_a) repeats ℓ_g full-state iterations followed by ℓ_a grouped iterations. The groups are rebuilt before each grouped phase. Their values are copied back to individual states before the next full-state phase.

After each full-state sweep, I record the residual range $r_t = \max_s (TV_t - V_t)(s) - \min_s (TV_t - V_t)(s)$. The tested feedback rule is

$$\varepsilon_t^{\text{RESIDUAL}} = \max\{\varepsilon_{\min}, cr_t\}, \quad c = 0.084, \quad \varepsilon_{\min} = 0.05. \quad (2)$$

The constant was fixed at the preregistered (2, 5) mix, to align the first residual-controlled width with the coarsest fixed baseline: there the first grouped phase follows $\ell_g = 2$ common opening sweeps, after which $r = 5.93429$ deterministically, so $0.084r = 0.49848 \approx 0.5$. The opening sweeps are ordinary value iteration, so this width is available before the feedback method is run. It changes with ℓ_g , giving $\varepsilon_0^{\text{RESIDUAL}} = 0.400, 0.499$, and 0.538 at (5, 2), (2, 5), and (1, 20). The main control never reads r_t :

$$\varepsilon_j^{\text{GEOMETRIC}} = \max \left\{ \varepsilon_{\min}, \varepsilon_0^{\text{RESIDUAL}} \left(\frac{\varepsilon_{\min}}{\varepsilon_0^{\text{RESIDUAL}}} \right)^{j/(C-1)} \right\}, \quad (3)$$

over grouped phases j , with $(\varepsilon_0^{\text{RESIDUAL}}, C)$ set per mix to (0.400, 7), (0.499, 13), and (0.538, 14): the starting width, the minimum, and the cycle at which RESIDUAL reaches the minimum. GEOMETRIC therefore matches the start and end of RESIDUAL's schedule at every mix. The only difference is that RESIDUAL uses the residual to choose each width, whereas GEOMETRIC follows a schedule fixed in

advance. A method with fixed $\varepsilon = 0.05$ tests whether gradual refinement is useful at all. Results for slower geometric schedules and coarser fixed widths are included in the supplementary material.

$\varepsilon_0^{\text{RESIDUAL}}$ is predictable in advance but C is not, so GEOMETRIC obtains it from one RESIDUAL run. At the two added mixes, GEOMETRIC is therefore a matched comparison, not a preregistered one. This adjustment changes only the comparison method and makes its schedule more similar to RESIDUAL, which makes any advantage for feedback harder to find. The preregistered control used (0.4985, 14) for every mix. At (2, 5), that schedule agrees with the matched control to within one cycle. Section 5 shows the effect of using the unmatched schedule at the other mixes.

Why use a geometric comparison? The start and end of GEOMETRIC are matched to RESIDUAL, and a standard result determines the general rate between them.

Proposition 1 (contraction of the residual range, 5, §6.6). *During the full-state sweeps of any schedule, $r_{t+1} \leq \gamma r_t$. Hence $\varepsilon^{\text{RESIDUAL}} \leq \max\{\varepsilon_{\min}, \varepsilon_0^{\text{RESIDUAL}} \gamma^{n_g}\}$, where n_g counts full-state sweeps since the first grouped phase, and the width reaches its minimum within $\lceil \log(\varepsilon_{\min}/\varepsilon_0^{\text{RESIDUAL}}) / \log \gamma \rceil$ of them.*

This contraction result is standard [5, §6.6]; I do not claim it as new. Appendix A includes the proof to make the constants explicit. Its consequence for ε is what matters here. Because the feedback rule reads a residual that shrinks geometrically during full-state sweeps, a schedule fixed in advance can reproduce its general rate. The proposition does not cover grouped-update phases, which also change V . It therefore provides an upper bound only while the residual is still setting the width. The bound holds at every recorded step in that period for all three mixes. Per cycle, $\varepsilon^{\text{RESIDUAL}}$ changes by factors of 0.698, 0.829, and 0.829, compared with upper bounds γ^{ℓ_g} of 0.774, 0.903, and 0.950. Grouped updates account for the gap between these numbers. For that reason, the exact cycle C must be measured rather than calculated from γ^{ℓ_g} alone.

Inventory MDP, protocol, and metrics. A state is the inventory vector $q \in \{-10, \dots, 10\}^3$, giving $|\mathcal{S}| = 21^3 = 9,261$. Five choices of quote aggressiveness trade fill probability against spread, and the one-period cost combines spread revenue with correlated inventory risk (Appendix C). I use $\gamma = .95$ and rescale costs so $\|V^*\|_\infty = 100$. This is a small market-making-style control model, not a calibrated market simulator. Each method runs for 10,000 iterations at each mix. The MDP itself is deterministic, so seeds 0–19 vary only which state is sampled within each group. The same seeds are used for all methods. The main accuracy measure is $\text{err}_\infty = \|V - V^*\|_\infty$, the largest absolute error across states. I also evaluate the policy chosen from V using $\|V^{\pi_V} - V^*\|_\infty$. Plots against computation use counted Bellman backups $B = n_g |\mathcal{S}| + \sum_t K_t$, where K_t is the number of occupied groups. This count favors aggregation because it omits the residual scan and the $(n_r + n_l)|\mathcal{S}|$ state operations used to rebuild and expand groups. Paired differences are summarized by their median and a 95% percentile bootstrap interval based on 10,000 resamples. Before the experiment, I defined differences in $[-0.02, 0.02]$ on the 100-normalized value scale as too small to matter in practice.

4 Results

Once the residual and geometric rules reach $\varepsilon_{\min}$ their error curves are visually indistinguishable at every mix (Figure 1, Appendix B). Table 1 shows the small numerical differences. At all three mixes, the 95% interval for residual minus matched geometric both contains zero and stays within ± 0.02 . The median differences are 1.7×10^{-7} , 5.1×10^{-7} , and -0.00201 . Thus, when the comparison has the same starting width, minimum width, and time to that minimum, using the residual does not improve the result after 10,000 iterations.

Value error and policy quality need not agree. At (2, 5), the residual and matched geometric methods have the same value error to seven decimal places, yet their policy losses differ by 19%. At (1, 20), the residual method is worse than fixed-0.05 on both measures: its value error is 0.01250 higher (95% CI [0.00745, 0.01561]), and its policy loss is 0.03529 higher. Neither measure supports using feedback in this experiment. Within each mix, the median computation at the end differs by at most 0.13%, although rebuilding and expanding the groups adds 39–96% more operations for the residual method. Full-state VI reaches its 10^{-10} tolerance in 454 sweeps, or 4.20M counted backups. Aggregation is therefore not competitive on this example; the experiment studies one approximation method rather than proposing a better solver.

Table 1: Results after 10,000 iterations over 20 paired seeds. R/G/F denote residual, matched geometric, and fixed $\varepsilon = .05$. Policy loss and B(M), millions of counted backups, follow the same order. The interval summarizes the 20 within-seed differences, rather than comparing the three separately computed medians.

Mix	$\text{err}_\infty(\text{R})$	$\text{err}_\infty(\text{G})$	$\text{err}_\infty(\text{F})$	R-G (95% CI)	Policy loss R/G/F	B(M) R/G/F
(5, 2)	.05001	.05001	.05001	$1.7\text{e-}7$ [$-2.8\text{e-}7$, $9.0\text{e-}7$]	.00210/.00210/.00210	67.685/67.684/67.686
(2, 5)	.13882	.13882	.13792	$5.1\text{e-}7$ [$-9.6\text{e-}5$, $2.0\text{e-}5$]	.01518/.01280/.01280	30.214/30.206/30.195
(1, 20)	.20343	.20115	.19093	-.00201 [$-.00320$, .00541]	.05178/.05178/.01649	9.185/9.186/9.196

Table 2: Median within-seed difference in value error at fixed compute budgets, shown as a percentage of the comparison method’s error. Negative values favor the residual method. RESIDUAL–FIXED measures the effect of gradually narrowing the groups; RESIDUAL–GEOMETRIC measures the additional effect of using feedback. * marks a 95% bootstrap interval that excludes zero. See Appendix D for details.

Mix	Contrast	10^5	3×10^5	10^6	3×10^6	final
(5, 2)	RESIDUAL–FIXED	+0.0	+0.2*	−0.1	−0.0	+0.0
(5, 2)	RESIDUAL–GEOMETRIC	+0.0	+0.0	−0.2*	+0.0	+0.0
(2, 5)	RESIDUAL–FIXED	+0.1	−23.5*	−14.1*	+0.4*	+0.6*
(2, 5)	RESIDUAL–GEOMETRIC	−0.1	−0.1	+0.2*	−0.0	+0.0
(1, 20)	RESIDUAL–FIXED	−18.8*	−5.1*	−5.4*	+10.2*	+6.5*
(1, 20)	RESIDUAL–GEOMETRIC	+4.5*	+10.8*	+3.4*	−1.4	−1.0

Separating the schedule from the feedback. The result after 10,000 iterations shows whether feedback changes the final estimate, but it hides the effect of gradually narrowing the groups. By then, every scheduled method has used $\varepsilon_{\min}$ for most of the run. At (5, 2), for example, the residual and fixed methods end only 3.8×10^{-8} apart. Table 2 instead compares the same runs at four earlier compute budgets, while the widths are still changing. It reports differences relative to the comparison method because a fixed $\pm .02$ threshold is not informative when both errors are near 10.

Among differences whose 95% intervals exclude zero, feedback’s largest advantage over the matched schedule is 0.2% of the comparison error, and its largest disadvantage is 10.8%. By contrast, gradual refinement changes the error by about 23% at one budget. Its direction also changes: at (1, 20), the changing width is 18.8% better than a fixed width after 10^5 backups, but 10.2% worse after 3×10^6 . The schedule can therefore help or hurt, while feedback adds no consistent benefit, in line with Proposition 1. The large RESIDUAL–FIXED differences occur while the error is dropping quickly, so a fixed compute budget may stop just before or after an update. Their direction and approximate size are stable, but their exact percentages are not.

The timing-based primary measure was inconclusive. The original primary comparison, (2, 5) at 20 ms, gave a residual-minus-geometric median of -0.0417 with 95% CI $[-0.0459, 0.0011]$, which is inconclusive under the predefined equivalence rule. Repeated runs changed the interval because the error was falling quickly at 20 ms. Small changes in machine load therefore changed which iteration finished in time. I report rather than replace that outcome. The comparisons at fixed numbers of backups were added after this problem appeared and were not preregistered.

5 What the negative result shows

The schedules must be matched on the right scale. A preplanned comparison must use the same unit that triggers changes in width. Here that unit is the number of grouped-update phases, not the total number of iterations. If the slow geometric schedule is copied unchanged to (1, 20), it does not reach its minimum width and ends ten times worse than every other method. Changing only its number of cycles removes this difference (Appendix E).

A mismatched schedule creates an apparent effect. When fixed at (0.4985, 14) for every mix, the preregistered geometric method differs from residual at (5, 2) by .00632, 95% CI [.006319, .006327]: narrow enough to exclude zero, with all 20 seeds favoring the geometric method. That difference is real and measured precisely, but it comes from the mismatch. At this mix, the geometric method starts with groups 25% wider than the residual method and takes twice as long to reach the minimum

width. Matching those two features reduces the difference to 1.7×10^{-7} , with an interval containing zero. When simulation variance is low, even a small mismatch can produce a precise difference that is incorrectly attributed to feedback.

6 Limitations and conclusion

One deterministic instance, one residual statistic, one minimum width, and one feedback constant support no claim about other MDPs; the mixes, budget ladder, and backup measurements were added after the preregistered experiment, and the backup count omits the cost of rebuilding and expanding groups. Feedback may still help in settings not covered by Proposition 1, such as when residuals differ sharply across regions, the best minimum width is unknown, or computation stops before refinement is complete.

Within the scope of this experiment, residual feedback never improves on a preplanned schedule with the same start, minimum, and duration. Gradually narrowing the groups does matter, changing error by about 23% at one matched budget, but it helps at some budgets and hurts at others. To credit feedback for an improvement, experiments must compare it with a method that follows the same general schedule and must measure performance at several compute budgets.

References

- [1] Ward Whitt. Approximations of dynamic programs, I. *Mathematics of Operations Research*, 3(3):231–243, 1978.
- [2] Dimitri P. Bertsekas and David A. Castanon. Adaptive aggregation methods for infinite horizon dynamic programming. *IEEE Transactions on Automatic Control*, 34(6):589–598, 1989.
- [3] Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a unified theory of state abstraction for MDPs. In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics*, 2006.
- [4] Guanting Chen, Johann Demetrio Gaebler, Matt Peng, Chunlin Sun, and Yinyu Ye. An adaptive state aggregation algorithm for Markov decision processes. *arXiv preprint arXiv:2107.11053*, 2021.
- [5] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, New York, 1994.
- [6] Benjamin Van Roy. Performance loss bounds for approximate value iteration with state aggregation. *Mathematics of Operations Research*, 31(2):234–244, 2006.
- [7] Satinder P. Singh, Tommi Jaakkola, and Michael I. Jordan. Reinforcement learning with soft state aggregation. In *Advances in Neural Information Processing Systems 7*, pages 361–368, 1995.
- [8] Rémi Munos and Andrew Moore. Variable resolution discretization in optimal control. *Machine Learning*, 49(2–3):291–323, 2002.
- [9] Orso Forghieri, Hind Castel, Emmanuel Hyon, and Erwan Le Pennec. Progressive state space disaggregation for infinite horizon dynamic programming. In *Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling*, pages 221–229, 2024.
- [10] Ronald J. Williams and Leemon C. Baird III. Tight performance bounds on greedy policies based on imperfect value functions. Technical Report NU-CCS-93-14, Northeastern University, 1993.
- [11] E. F. Arruda, M. D. Fragoso, and J. B. R. do Val. Approximate dynamic programming via direct search in the space of value function approximations. *European Journal of Operational Research*, 211(2):343–351, 2011.
- [12] Andrew W. Moore and Christopher G. Atkeson. Prioritized sweeping: Reinforcement learning with less data and less real time. *Machine Learning*, 13(1):103–130, 1993.
- [13] David Wingate and Kevin D. Seppi. Prioritization methods for accelerating MDP solvers. *Journal of Machine Learning Research*, 6:851–881, 2005.
- [14] Matthieu Geist, Bilal Piot, and Olivier Pietquin. Is the Bellman residual a bad proxy? In *Advances in Neural Information Processing Systems 30*, pages 3205–3214, 2017.

A Proof of Proposition 1

The following standard result says that the Bellman operator shrinks differences when they are measured by their range [5, §6.6]. The proof is included to make the constants in the bound explicit.

Fix V, U and a state s , and let a^* attain the minimum in $(TU)(s)$. Evaluating the bracket of (1) at a^* rather than at the minimizer for V gives

$$\begin{aligned} (TV)(s) &\leq c(s, a^*) + \gamma \sum_{s'} P(s' | s, a^*) V(s') = (TU)(s) + \gamma \sum_{s'} P(s' | s, a^*) (V - U)(s') \\ &\leq (TU)(s) + \gamma \max_{s'} (V - U)(s'), \end{aligned}$$

and the symmetric argument, taking a^* to attain the minimum in $(TV)(s)$, gives $(TV)(s) \geq (TU)(s) + \gamma \min_{s'} (V - U)(s')$. Both bounds hold at every s , so $(TV - TU)(s)$ lies in $[\gamma \min(V - U), \gamma \max(V - U)]$. Defining $\text{span}(x) = \max(x) - \min(x)$, it follows that

$$\text{span}(TV - TU) \leq \gamma \text{span}(V - U).$$

During a full-state phase $V_{t+1} = TV_t$, so $r_{t+1} = \text{span}(TV_{t+1} - V_{t+1}) = \text{span}(TV_{t+1} - TV_t) \leq \gamma \text{span}(V_{t+1} - V_t) = \gamma \text{span}(TV_t - V_t) = \gamma r_t$. Iterating over the n_g full-state sweeps taken since the first grouped phase and substituting into (2) gives the stated bound, and the minimum width is reached once $\varepsilon_0^{\text{RESIDUAL}} \gamma^{n_g} \leq \varepsilon_{\min}$.

The proposition does not cover a grouped phase. Such a phase copies updates from representative states to their groups instead of applying T to every state, so it can increase the residual range. The bound therefore applies only while the rule is still setting the width. It holds empirically at every recorded step in that period for all three mixes.

B Error over computation

Rules reaching $\varepsilon_{\min}$ have similar error floors — inventory $N = 3, Q = 10, \gamma = 0.95$

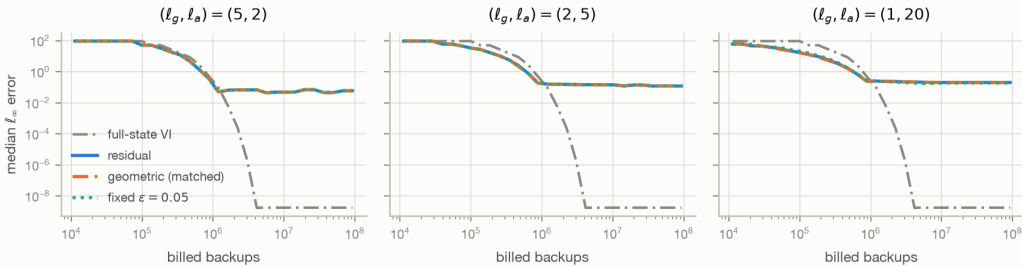


Figure 1: Median value error against counted Bellman backups. Final estimates are displayed beyond each method’s 10,000-iteration limit. Residual, fast geometric, and fixed-0.05 nearly coincide within each mix. The deterministic VI curve is repeated for reference; its convergence check stops after 454 sweeps (4.20M backups) though the plotted diagnostic continues to the common stopping point.

The flat right-hand portions repeat the final values; they do not represent continued computation. Each method stops after 67.7M, 30.2M, or 9.2M counted backups at (5, 2), (2, 5), or (1, 20), respectively. The plots carry the last value forward so that the three panels can share an axis. Budgets beyond a method’s stopping point therefore cannot be compared fairly, which is why the list in Appendix D stops at 3×10^6 .

C Inventory instance

The five joint quote-aggressiveness actions have fill probabilities (.1, .2, .4, .6, .8) and spreads (1, .8, .5, .3, .15), so aggressive quotes fill often at a worse price. The one-period cost is

$$c(q, a) = 0.02 q^\top \Sigma q - \delta_a f_a L(q),$$

where δ_a and f_a are the spread and fill probability of action a , Σ has unit diagonal and correlation 0.5 so inventory risk is correlated across the three assets, and $L(q)$ removes fills that would leave the box $\{-10, \dots, 10\}^3$. Costs are rescaled after solving so that $\|V^*\|_\infty = 100$, which puts ε , the equivalence margin, and every reported error on one interpretable scale.

D How the budget comparisons are calculated

Each cell of Table 2 is the median over 20 seeds of the within-seed difference divided by the comparison method’s error for that seed at the same budget. Calculating the ratio before taking the median preserves the pairing. Errors are read as a step function of counted backups: each budget uses the most recent completed update. The list stops at 3×10^6 because the cheapest mix, (1, 20), performs only 9.2M backups in total. Later budgets would compare a repeated final value with continued computation. The supplementary material reports absolute differences and 10,000-resample percentile-bootstrap intervals for both the absolute and relative medians.

E Adjusting the slow schedule across update mixes

The preregistered slow geometric method used 1,429 grouped-update cycles, exactly the number induced by 10,000 iterations at cycle length seven. Transplanted unchanged to (1, 20), whose cycle length is 21, it receives only 477 grouped phases and is still narrowing at the stopping point. It ends at $\varepsilon = .232$ and error 1.59, an order of magnitude worse than every other method. Recalculating only its cycle count to 477, changing nothing else, ends it at $\varepsilon = .05024$ and error .209, back in the neighborhood of residual and fast geometric. The poor result came from an unfinished schedule, not from the lack of feedback. For this reason, mixes other than the preregistered (2, 5) use the schedule with the corrected cycle count.